

Diseño de Sistemas de información

SOLID

¿Qué son los principios SOLID ?

Los principios SOLID son un conjunto de cinco principios de diseño de software que buscan promover la creación de código limpio, mantenible y extensible. Estos principios fueron propuestos por **Robert C. Martín** y son ampliamente utilizados en la ingeniería de software.

¿Por qué importan?

- Reducen el acoplamiento entre componentes
- Facilitan el mantenimiento y la evolución del software
- Hacen el código más testeable y reutilizable
- Anticipan el cambio en lugar de temerle

S Single Responsibility

O Open/Closed

L Liskov Substitution

I Interface Segregation

D Dependency Inversion

El acrónimo, en una frase cada uno



Responsabilidad Única

Una clase, una razón para cambiar.



Abierto / Cerrado

Extender sin modificar lo existente.



Sustitución de Liskov

Una subclase no debe romper el contrato.



Segregación de Interfaces

Interfaces chicas y específicas.



Inversión de Dependencias

Depender de abstracciones, no de detalles.

SRP - Principio de Responsabilidad Única

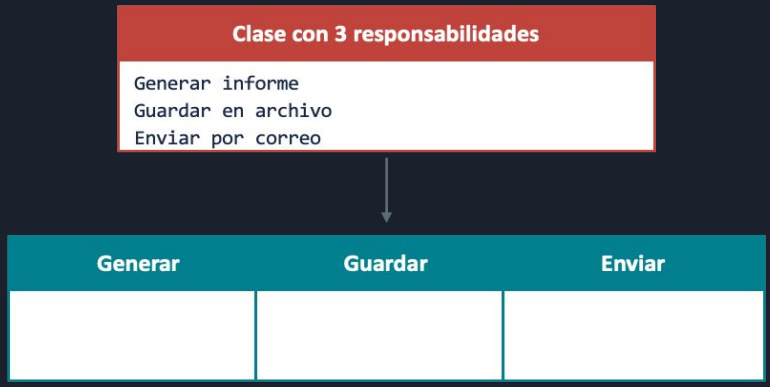
Cada clase o módulo debe encargarse de una única responsabilidad dentro del sistema. Una clase debe tener una SOLA razón para cambiar: si cambia por múltiples motivos, está violando SRP.

Se relaciona con un concepto que ya conocés:
COHESIÓN

Ventajas

- Facilita el mantenimiento
- Evita errores colaterales
- Mejora la reutilización
- Favorece las pruebas unitarias

Divide y vencerás



Una responsabilidad clara por clase → una sola razón para cambiar.

SRP: Antes y después

✗ Sin aplicar SRP

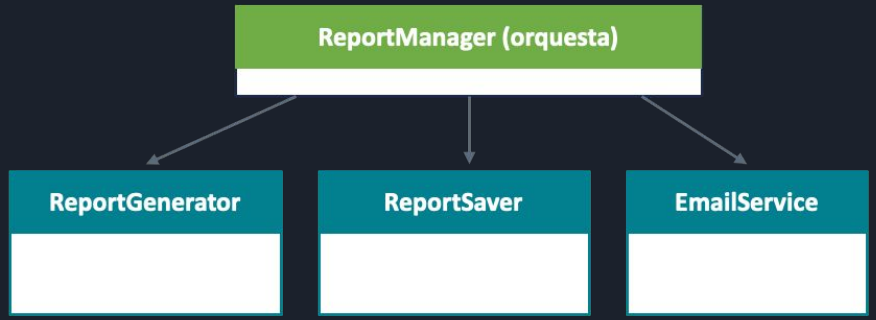
ReportManager

```
+ GenerateReport()
+ SaveToFile(content)
+ SendEmail(email)
```

✗ Viola el principio

Tres razones distintas para modificar esta clase: cambia el formato del informe, cambia el destino de guardado o cambia el proveedor de email.

✓ Aplicando SRP



✓ Aplica el principio

Cada clase cambia por un único motivo. ReportManager solo coordina — no implementa nada.

SRP: ejemplo

```
public class ReportManager
{
    public void GenerateReport()
    {
        // Genera el informe
        Console.WriteLine("Generando el informe...");
    }

    public void SaveToFile(string content)
    {
        // Guarda el informe en un archivo
        File.WriteAllText("reporte.txt", content);
    }

    public void SendEmail(string email)
    {
        // Envía el informe por correo
        Console.WriteLine($"Enviando el informe a {email}'
    }
}
```

Sin aplicar SRP

SRP: ejemplo

```
public class ReportGenerator
{
    public string GenerateReport()
    {
        return "Este es un informe generado.";
    }
}

public class ReportSaver
{
    public void SaveToFile(string content)
    {
        File.WriteAllText("reporte.txt", content);
    }
}

public class EmailService
{
    public void SendEmail(string email, string content)
    {
        Console.WriteLine($"Enviando el informe a {email}"
    }
}
```

```
public class ReportManager
{
    private readonly ReportGenerator _generator;
    private readonly ReportSaver _saver;
    private readonly EmailService _emailService;

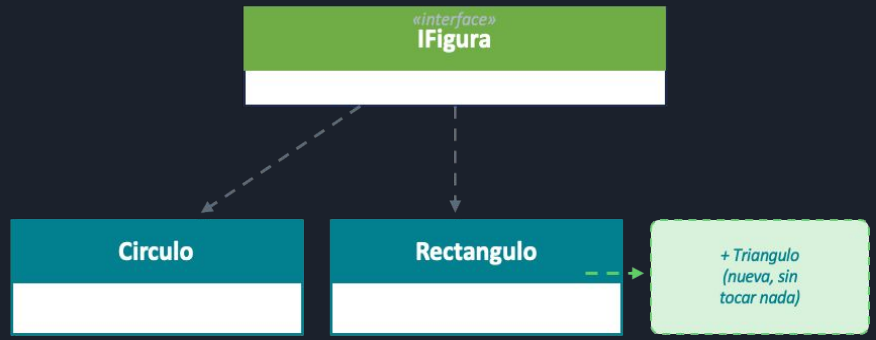
    public ReportManager()
    {
        _generator = new ReportGenerator();
        _saver = new ReportSaver();
        _emailService = new EmailService();
    }

    public void ProcessReport(string email)
    {
        var report = _generator.GenerateReport();
        _saver.SaveToFile(report);
        _emailService.SendEmail(email, report);
    }
}
```

OCP: principio de abierto/cerrado

Las entidades de software (clases, módulos, etc.) deben estar abiertas para la extensión pero cerradas para la modificación.

Puedes extender el comportamiento de una clase o módulo sin necesidad de modificar su código existente.



Cada figura implementa el contrato — agregar una nueva no requiere modificar las existentes.

Ventajas

- Reduce el impacto de cambios en código existente
- Evita romper funcionalidades ya probadas
- Favorece el uso de abstracciones e interfaces
- Fomenta la reutilización y mantenibilidad

OCP: Antes y después

✗ Sin aplicar OCP

CalculadoraDeArea

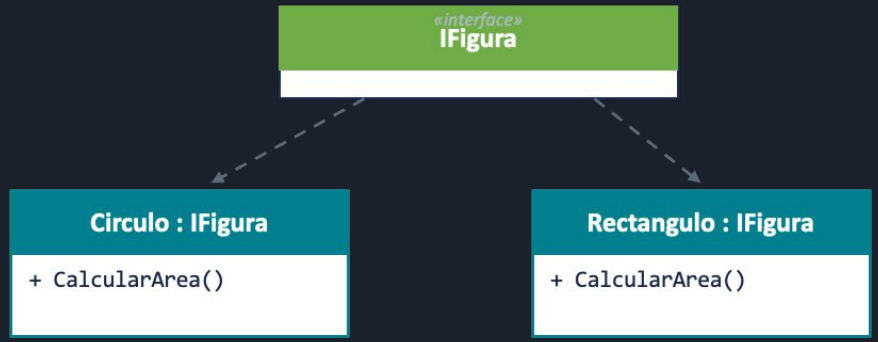
```
+ CalcularArea(figura: object)
```

```
if (figura is Circulo c) return 3.14 * c.Radio * c.Radio;
else if (figura is Rectangulo r) return r.Ancho * r.Alto;
else throw new ArgumentException("no soportada");
// cada figura nueva obliga a MODIFICAR este método
```

✗ Viola el principio

El método crece y se vuelve más frágil con cada figura nueva; corre riesgo de romper los casos ya probados.

✓ Aplicando OCP



✓ Aplica el principio

Cada figura sabe calcular su propia área. Para sumar una figura nueva, se crea una clase — nunca se toca una existente.

OCP: ejemplo

```
public class Circulo
{
    public double Radio { get; }

    public Circulo(double radio)
    {
        Radio = radio;
    }
}

public class Rectangulo
{
    public double Ancho { get; }
    public double Alto { get; }

    public Rectangulo(double ancho, double alto)
    {
        Ancho = ancho;
        Alto = alto;
    }
}
```

Sin aplicar OCP

```
public class CalculadoraDeArea
{
    public double CalcularArea(object figura)
    {
        if (figura is Circulo c)
        {
            return 3.14 * c.Radio * c.Radio;
        }
        else if (figura is Rectangulo r)
        {
            return r.Ancho * r.Alto;
        }
        else
        {
            throw new ArgumentException("Figura no soport
        }
    }
}
```

Sin aplicar OCP

OCP: ejemplo

```
// Creamos la clase Rectangulo que implementa la interfaz
public class Rectangulo : IFigura
{
    public double Ancho { get; }
    public double Alto { get; }

    public Rectangulo(double ancho, double alto)
    {
        Ancho = ancho;
        Alto = alto;
    }

    public double CalcularArea()
    {
        return Ancho * Alto;
    }
}
```

```
// Definimos una interfaz para garantizar que todas las f
public interface IFigura
{
    double CalcularArea();
}

// Creamos la clase Circulo que implementa la interfaz IF
public class Circulo : IFigura
{
    public double Radio { get; }

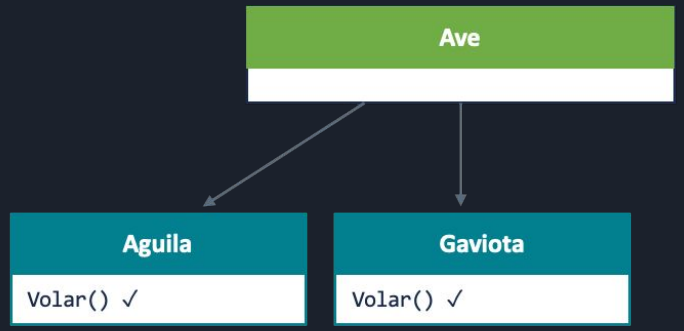
    public Circulo(double radio)
    {
        Radio = radio;
    }

    public double CalcularArea()
    {
        return Math.PI * Radio * Radio;
    }
}
```

LSP: principio de sustitución de Liskov

Los objetos de una clase derivada deben poder sustituir a los de la clase base sin afectar la integridad del programa. (Bárbara Liskov, 1987).

Imagine un control remoto universal diseñado para funcionar con cualquier televisor. Si un nuevo modelo de TV no responde a los mismos comandos, el control deja de ser útil. El **Principio de Sustitución de Liskov** es como una garantía de que cualquier 'televisor' (o clase derivada) va a funcionar correctamente con el 'control remoto' (o clase base).




```
Ave a = new Aguila();
a.Volar();  // funciona sin sorpresas
```

Cualquier subtipo de Ave debe poder "reemplazar" a Ave sin romper el comportamiento esperado.

LSP: ventajas

- Evita errores sutiles al usar herencia.
- Mantiene el comportamiento coherente al sustituir objetos.
- Fortalece el polimorfismo seguro.
- Mejora la reutilización sin romper funcionalidades existentes.
- Fomenta un contrato claro entre superclase y subclases.

LSP: Antes y después

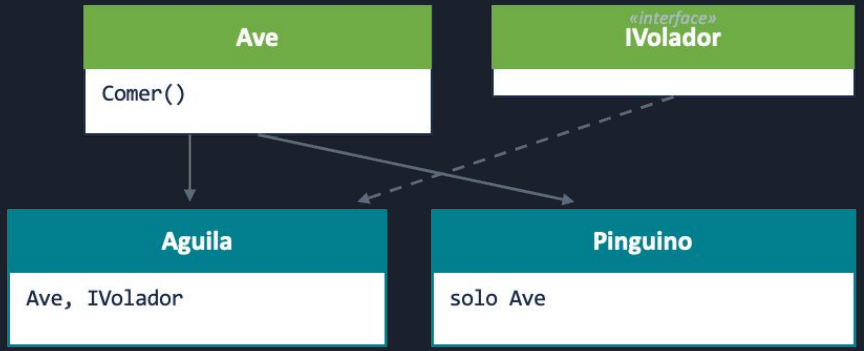
✗ Sin aplicar LSP



✗ Viola el principio

Pinguino "es un" Ave en el modelo, pero no cumple el contrato: sustituirlo rompe cualquier código que llame a Volar().

✓ Aplicando LSP



✓ Aplica el principio

Pinguino ya no promete volar: implementa Ave (Comer) pero no IVolador. Ningún cliente puede "romperse" invocando Volar() sobre un pingüino.

LSP: ejemplo

- Este código cumple con el principio LSP.

```
Ave a = new Ave();
Ave b = new Aguila();
Aguila c = new Aguila();

a.Volar();
b.Volar();
c.Volar();


public class Ave
{
    public virtual void Volar()
    {
        Console.WriteLine("El ave, vuela.");
    }
}

public class Aguila:Ave
{
    public override void Volar()
    {
        Console.WriteLine("El águila, vuela muy alto.");
    }
}
```

LSP: ejemplo

- Ahora bien, si ahora creamos la clase Pingüino que NO puede volar, entonces estaríamos violando el principio.

```
public class Pinguino : Ave
{
    public override void Volar()
    {
        throw new NotImplementedException("Penguins canno
    }
}
```

```
Ave miPinguino = new Pinguino();
unPinguino.Volar(); // Esto lanza una excepción, violando
```

LSP: ejemplo

- Se propone usar una interfaz para cumplir con el principio LSP.

```
Aguila unAguila = new Aguila();
unAguila.Volar(); // "El águila vuela muy alto."

Pinguino unPinguino = new Pinguino();
// unPinguino.Volar(); Error: No existe el método Volar e
```

```
public interface IVolador
{
    void Volar();
}

public class Ave
{
    public void Comer()
    {
        Console.WriteLine("El ave, está comiendo.");
    }
}

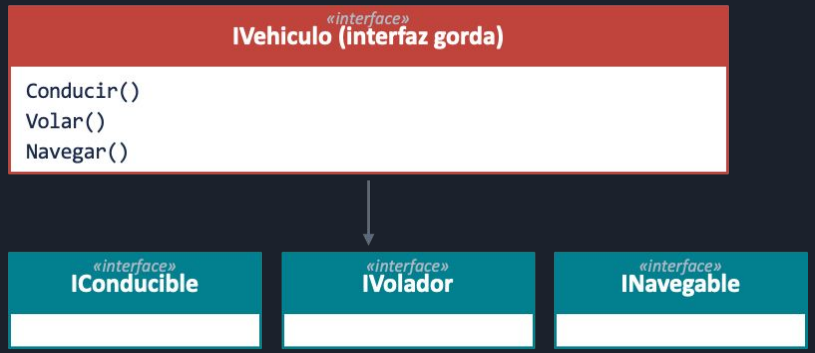
public class Aguila : Ave, IVolador
{
    public void Volar()
    {
        Console.WriteLine("El águila, vuela muy alto.");
    }
}

public class Pinguino : Ave
{
    // No implementamos IVolador.. porque los pingüinos n
    // excepto el de la película de Batman .)
}
```

ISP: principio de segregación de interfaces

Diseñar interfaces específicas y enfocadas, evitando contratos innecesarios o pesados. Una clase no debería verse obligada a depender de métodos que no utiliza.

Mejor muchas interfaces chicas y específicas, que una sola grande y genérica



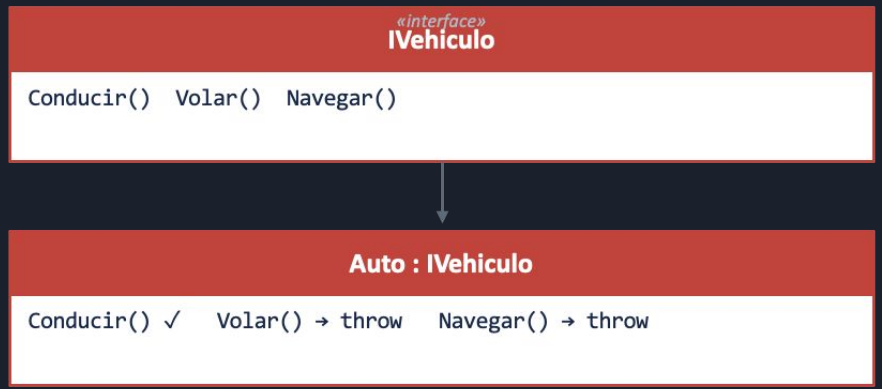
Se separa en contratos pequeños: cada clase implementa solo lo que necesita.

Ventajas

- Alta cohesión, menor acoplamiento.
- Mejora flexibilidad y mantenibilidad.
- Facilita cambios y evolución del sistema.

ISP: Antes y después

✗ Sin aplicar ISP



✗ Viola el principio

Auto queda "obligado" a implementar Volar() y Navegar(), métodos que no usa ni tiene sentido que use.

✓ Aplicando ISP



✓ Aplica el principio

Cada clase implementa solo la interfaz que necesita. Ninguna se ve forzada a tener métodos vacíos o excepciones "no implementado".

ISP: ejemplo

- Este código NO cumple con el principio ISP.

```
public interface IVehiculo {

    void Conducir();

    void Volar();

    void Navegar();

}
```

```
public class Auto : IVehiculo {

    public void Conducir() { Console.WriteLine("Conducien

    public void Volar() { throw new NotImplementedExcepti

    public void Navegar() { throw new NotImplementedExcep

}
```

ISP: ejemplo

- Rediseñamos el código para que cumpla con el principio ISP.

```
public interface IConducible { void Conducir(); }

public interface IVolador { void Volar(); }

public interface INavegable { void Navegar(); }
```

Y ahora las clases pueden implementar solo lo que necesitan:

```
public class Auto : IConducible {

public void Conducir() { Console.WriteLine("Conduciendo")

}

public class Avion : IVolador {

    public void Volar() { Console.WriteLine("Volando"); }

}

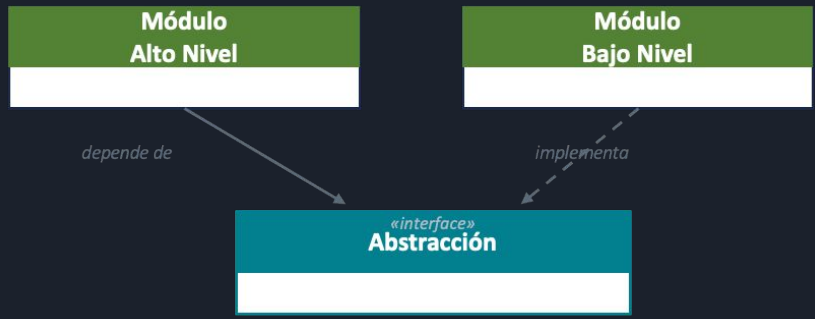
public class Barco : INavegable {

    public void Navegar() { Console.WriteLine("Navegando"

}
```

DIP: principio de inversión de dependencias

Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones. Las abstracciones no dependen de los detalles: los detalles dependen de las abstracciones.



Ambos módulos apuntan hacia la abstracción — nunca uno hacia el otro directamente. Por eso se llama "inversión".

Alto nivel: orquesta la lógica de negocios (ej. un controlador)

Bajo nivel: ejecuta detalles concretos (ej. acceso a bd, notificaciones)

Abstracción: interfaz o clase abstracta: un “contrato” sin implementación.

DIP: principio de inversión de dependencia

- DIP se implementa en la práctica mediante:
  - IoC (Inversión de control): el control sobre las dependencias lo tiene un contenedor o framework.
  - DI (Inyección de dependencias): se inyecta la implementación de una interfaz, en lugar de crearla directamente.

¿Qué es inversión de control (IoC)?

- El flujo de control del programa ya no lo maneja directamente tú código, sino que lo transfiere a un contenedor, framework o infraestructura externa.
- Se delega la responsabilidad de crear y gestionar objetos a una entidad externa.

```
public class Car {
    private Engine engine = new Engine(); // ¡acoplamiento fuerte!

    public void Start() {
        engine.Ignite();
    }
}
```

¿Qué es inversión de control (IoC)?

- Ejemplo aplicando IoC:

```
public class Car {
    private readonly IEngine engine;

    public Car(IEngine engine) {
        this.engine = engine; // El motor ya viene creado desde afuera
    }

    public void Start() {
        engine.Ignite();
    }
}
```

¿Qué es inyección de dependencia (DI)?

- Es una forma concreta de aplicar Inversión de control, en la que las dependencias de un objeto se "**inyectan**" desde el exterior.
- La clase ya no crea sus dependencias, simplemente las recibe.
- **Toda DI aplica IoC, pero no toda IoC usa DI** (por ejemplo, callbacks o eventos son IoC)

Inyección por constructor

```
public class Car {
    private readonly IEngine engine;

    public Car(IEngine engine) {
        this.engine = engine;
    }
}
```

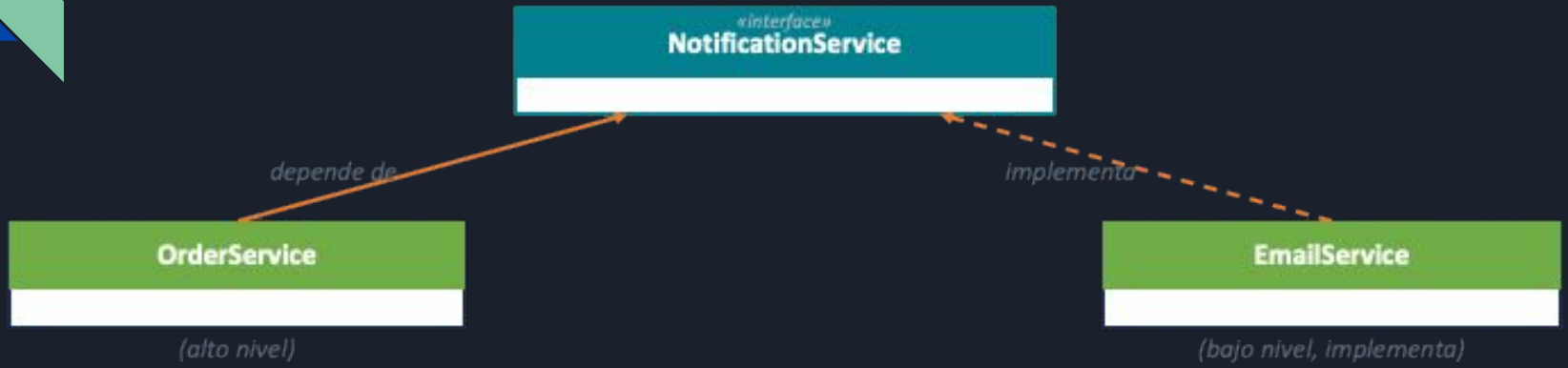
Inyección por propiedad

```
public class Car {
    public IEngine Engine { get; set; }
}
```

Inyección por método

```
public void StartCar(IEngine engine) {
    engine.Ignite();
}
```

DIP: ejemplo




```
interface NotificationService { void send(String message); }

class EmailService implements NotificationService {
  public void send(String message) { /* envía el email */ }
}

class OrderService {
  private NotificationService notifier;  // depende de la ABSTRACCIÓN
  public OrderService(NotificationService notifier) { this.notifier = notifier; } // DI por constructor
}
```

Principios SOLID: ejercicio